

# **SOLID Principles and Design Patterns in .NET**

**&**

## **Design Patterns**

**Name: Tushar Dev Singh**

**Introduction:** This project demonstrates the implementation of SOLID principles and commonly used design patterns in a .NET Console Application. The goal was to refactor the system to improve maintainability, scalability, and testability.

**SRP:** ReportGenerator handles report creation, ReportSaver handles saving.

**OCP:** IReportFormatter allows adding new formats (PDF, Excel) without modifying existing code.

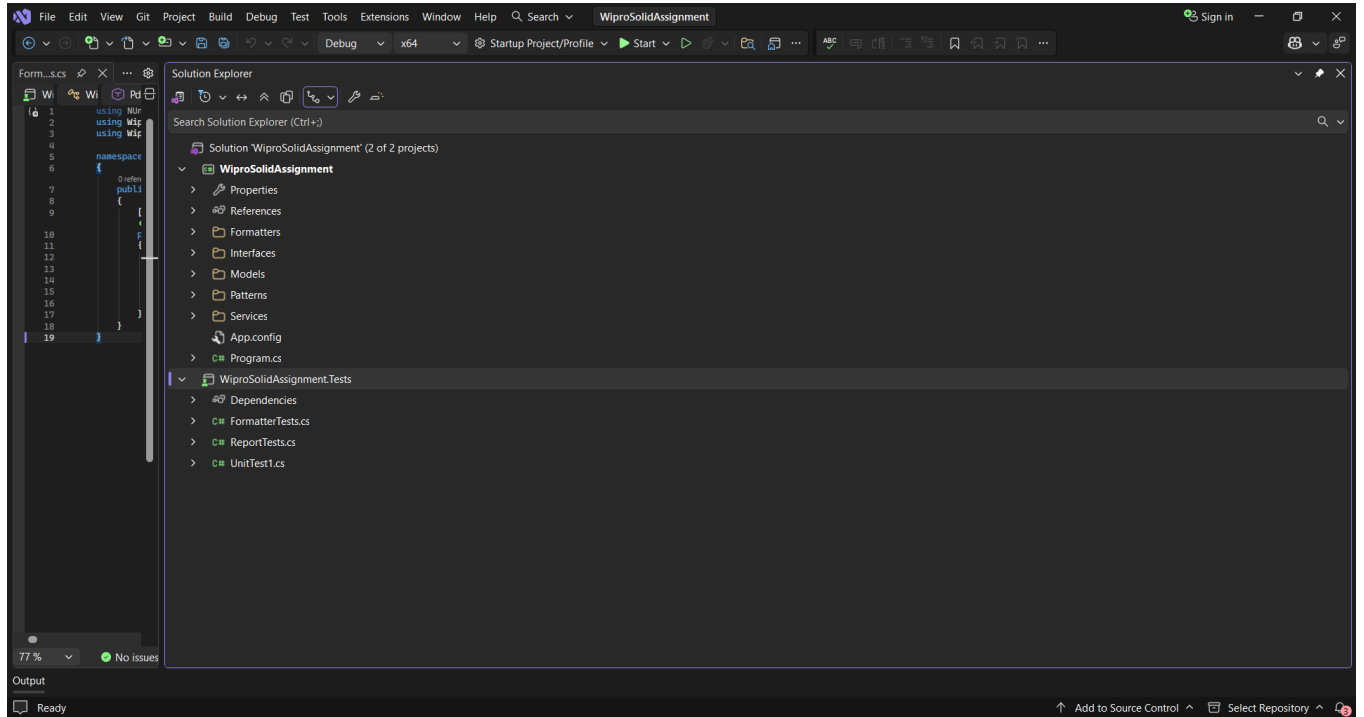
**LSP:** Derived report classes can replace base Report class without affecting behavior.

**ISP:** Interfaces are split so classes implement only required methods.

**DIP:** ReportService depends on abstractions using dependency injection.

**Design Pattern:** Singleton Logger ensures only one instance exists globally.

## Project Structure



## Program.cs:

```
using System;
using System.Collections.Generic;
using WiproSolidAssignment.Services;
using WiproSolidAssignment.Patterns;
using WiproSolidAssignment.Formatters;
namespace WiproSolidAssignment
{
    class Program
    {
        static void Main(string[] args)
        {
            var generator = new ReportGenerator();
            var saver = new ReportSaver();
            var service = new ReportService(generator, saver);
            service.CreateReport("Sales Report", "Revenue 10K");
            var formatter = new ReportFormatter(new PdfFormatter());
            Console.WriteLine(formatter.Format("Hello Report"));
            var logger = Logger.Instance;
            logger.Log("Application started");
            Console.ReadKey();
        }
    }
}
```

```
WiproSolidAssignment WiproSolidAssignment.Program
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7
8 using WiproSolidAssignment.Services;
9 using WiproSolidAssignment.Patterns;
10 using WiproSolidAssignment.Formatters;
11
12 namespace WiproSolidAssignment
13 {
14     0 references
15     class Program
16     {
17         0 references
18         static void Main(string[] args)
19         {
20             var generator = new ReportGenerator();
21             var saver = new ReportSaver();
22
23             var service = new ReportService(generator, saver);
24             service.CreateReport("Sales Report", "Revenue 10K");
25
26             var formatter = new ReportFormatter(new PdfFormatter());
27             Console.WriteLine(formatter.Format("Hello Report"));
28
29             var logger = Logger.Instance;
30             logger.Log("Application started");
31
32             Console.ReadKey();
33         }
34     }
35 }
```

## Logger.cs:

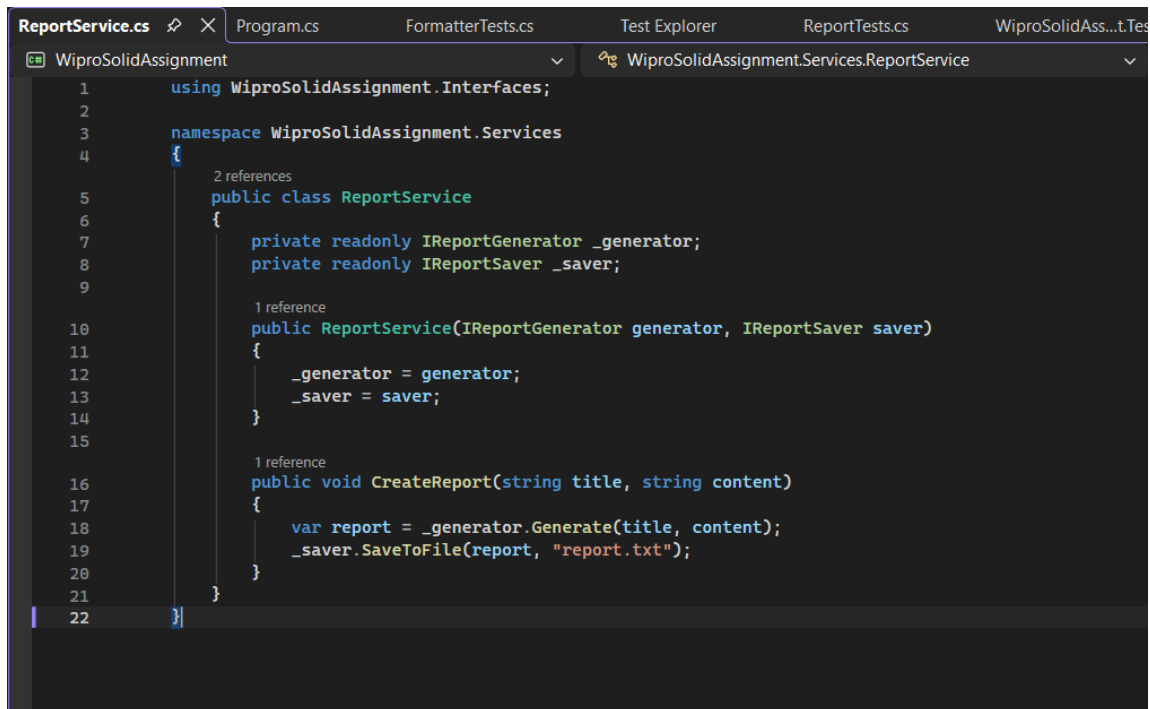
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System;
namespace WiproSolidAssignment.Patterns
{
    public sealed class Logger
    {
        private static Logger _instance;
        private static readonly object _lock = new object();
        private Logger() { }
        public static Logger Instance
        {
            get
            {
                lock (_lock)
                {
                    if (_instance == null)
                        _instance = new Logger();
                    return _instance;
                }
            }
        }
    }
}
```

```
    }  
}  
public void Log(string message)  
{  
    Console.WriteLine("LOG: " + message);  
}  
}  
}
```

```
WiproSolidAssignment WiproSolidAssignment.Patterns.Logger
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using System;
7
8  namespace WiproSolidAssignment.Patterns
9  {
10     7 references
11     public sealed class Logger
12     {
13         private static Logger _instance;
14         private static readonly object _lock = new object();
15
16         1 reference
17         private Logger() { }
18
19         3 references 1/1 passing
20         public static Logger Instance
21         {
22             get
23             {
24                 lock (_lock)
25                 {
26                     if (_instance == null)
27                         _instance = new Logger();
28                     return _instance;
29                 }
30             }
31
32             1 reference
33             public void Log(string message)
34             {
35                 Console.WriteLine("LOG: " + message);
36             }
37         }
38     }
39 }
```

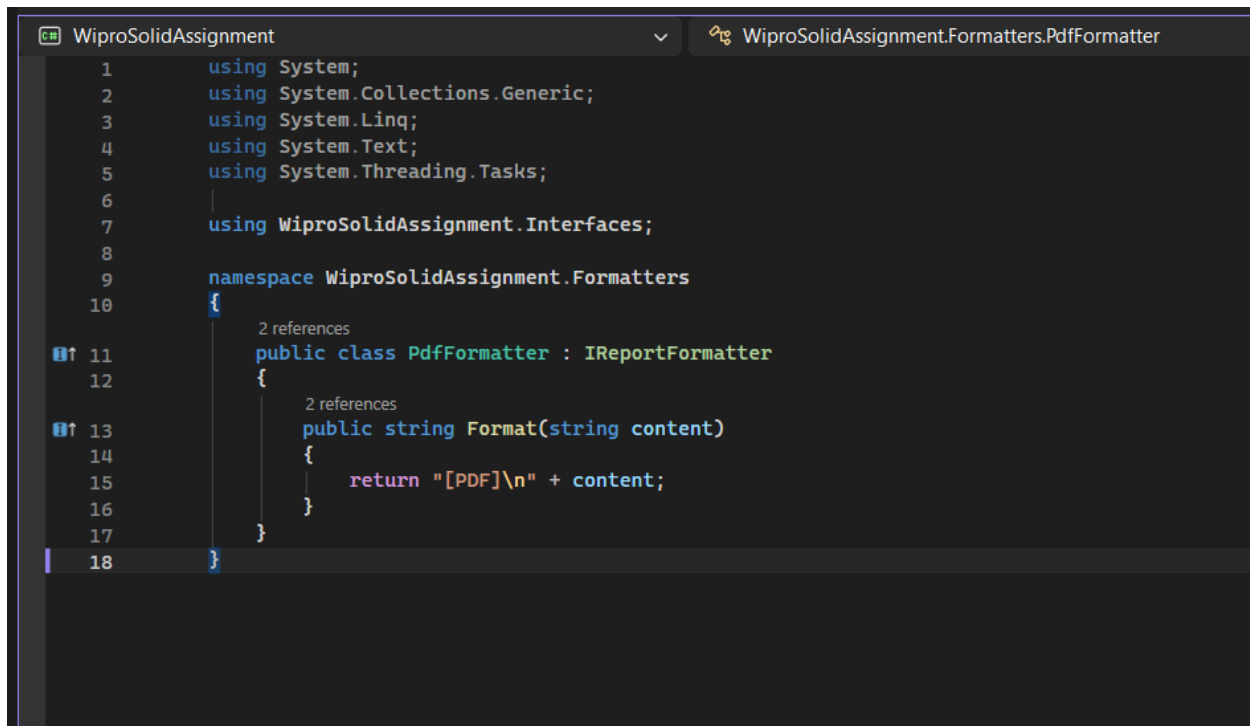
## ReportService.cs:

```
using WiproSolidAssignment.Interfaces;
namespace WiproSolidAssignment.Services
{
    public class ReportService
    {
        private readonly IReportGenerator _generator;
        private readonly IReportSaver _saver;
        public ReportService(IReportGenerator generator, IReportSaver saver)
        {
            _generator = generator;
            _saver = saver;
        }
        public void CreateReport(string title, string content)
        {
            var report = _generator.Generate(title, content);
            _saver.SaveToFile(report, "report.txt");
        }
    }
}
```



## PdfFormatter.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using WiproSolidAssignment.Interfaces;
namespace WiproSolidAssignment.Formatters
{
    public class PdfFormatter : IReportFormatter
    {
        public string Format(string content)
        {
            return "[PDF]\n" + content;
        }
    }
}
```





## Output

```
D:\Wipro2026\Day3lab\Wiprc x + v
[PDF]
Hello Report
LOG: Application started
|
```

## Test Cases output:

Test Explorer

3 3 0 Search (Ctrl+I)

Test discovery finished: 3 Tests found in 1.8 sec 1 Warning 0 Errors

| Test                           | Duration | Traits | Error Message |
|--------------------------------|----------|--------|---------------|
| WiproSolidAssignment.Tests (3) | 8 ms     |        |               |
| WiproSolidAssignment.Tests (3) | 8 ms     |        |               |
| FormatterTests (1)             | 5 ms     |        |               |
| LoggerTests (1)                | 1 ms     |        |               |
| ReportTests (1)                | 2 ms     |        |               |

Run Debug P

**Group Summary**

WiproSolidAssignment.1

Tests in group: 3

Total Duration: 8

**Outcomes**

3 Passed

## Design Pattern Implementation

**Singleton Pattern:** A Logger class was implemented using the Singleton pattern to ensure:

- Only one instance exists
- Global access point

Used throughout the application for logging.

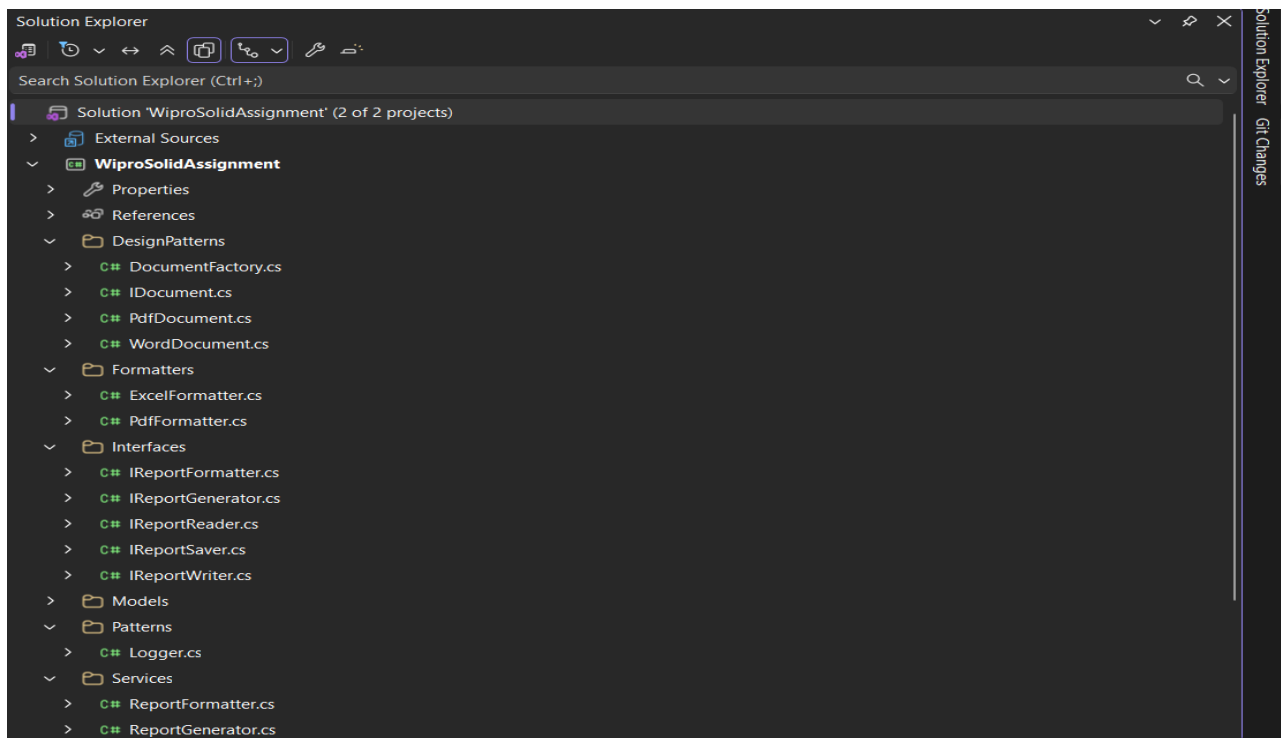
**Factory Pattern:** The Factory Pattern was implemented using:

- IDocument (interface)
- PdfDocument
- WordDocument
- DocumentFactory

The DocumentFactory class creates document objects based on input type without exposing object creation logic.

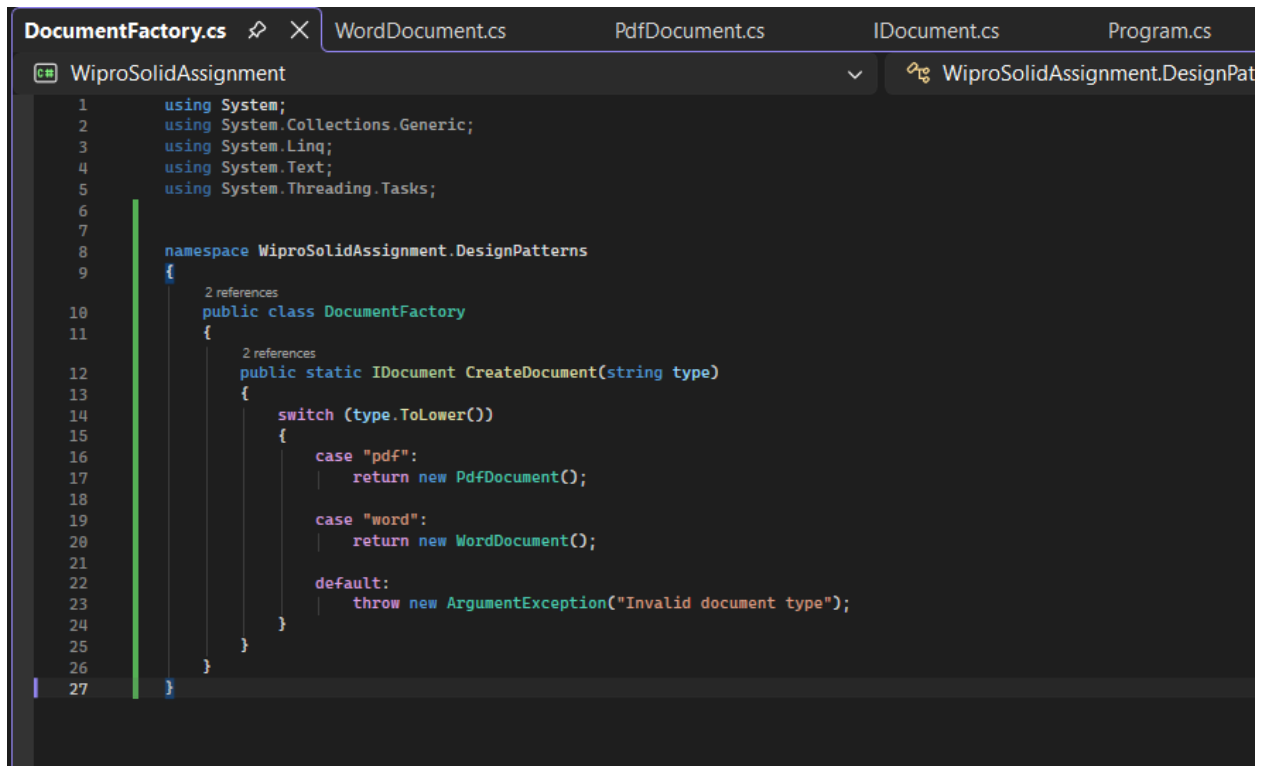
This improves:

- Flexibility
- Maintainability
- Extensibility



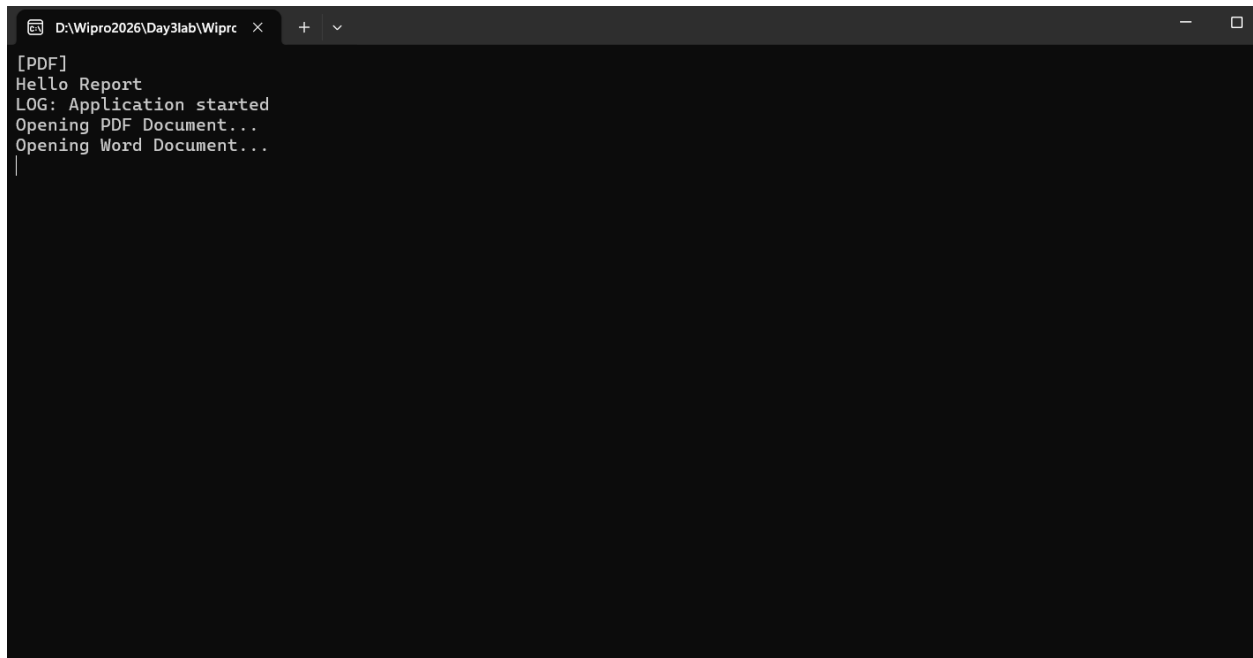
## Code Snippets

### DocumentFactory.cs:



```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7
8  namespace WiproSolidAssignment.DesignPatterns
9  {
10     2 references
11     public class DocumentFactory
12     {
13         2 references
14         public static IDocument CreateDocument(string type)
15         {
16             switch (type.ToLower())
17             {
18                 case "pdf":
19                     return new PdfDocument();
20                 case "word":
21                     return new WordDocument();
22                 default:
23                     throw new ArgumentException("Invalid document type");
24             }
25         }
26     }
27 }
```

## Output (Design Patterns)



The image shows a screenshot of a code editor window. The title bar at the top reads "D:\Wipro2026\Day3lab\Wiprc" followed by a close button (X) and a dropdown menu with a plus sign and a downward arrow. The editor area has a dark background and contains the following text:

```
[PDF]
Hello Report
LOG: Application started
Opening PDF Document...
Opening Word Document...
|
```

The text is displayed in a light gray font on the dark background. The cursor is positioned at the end of the last line, indicated by a vertical bar.